Transformers

Prepared By

SREEDEV C S

October 28, 2026

Contents

1 Introduction **1**

2 Tokenization **2**
- 2.1 What is Tokenization? . . . 2
- 2.2 Why Tokenization is Necessary? . . . 2
- 2.3 Tokenization Approaches . . . 2
  - 2.3.1 Word Tokenization . . . 2
  - 2.3.2 Character Tokenization . . . 3
  - 2.3.3 Subword Tokenization . . . 3
  - 2.3.4 Sentence tokenization . . . 3
  - 2.3.5 N-gram Tokenization . . . 4
- 2.4 Vocabulary . . . 4

3 Embeddings **5**
- 3.1 From Tokens to Numerical Representations . . . 5
- 3.2 One-Hot Encoding . . . 5
- 3.3 Dense Embeddings . . . 5
- 3.4 Positional Encoding . . . 6

4 Transformer Architecture Basics **7**
- 4.1 Overview . . . 7
- 4.2 How a Prompt Becomes an Output . . . 7
- 4.3 Self-Attention . . . 9
- 4.4 Illustrative Example . . . 9
- 4.5 Multi-Head Attention . . . 9
- 4.6 Feed Forward Networks . . . 9
- 4.7 Residual Connections and Layer Normalization . . . 10
- 4.8 Decoder-Only Architecture . . . 10
- 4.9 Autoregressive Text Generation . . . 10

5 Pretraining **11**
- 5.1 Definition . . . 11
- 5.2 Next Token Prediction . . . 11

5.3 Teacher Forcing 11
5.4 Training Data Sources 13
5.5 Computational Requirements 13

6 Finetuning **14**
6.1 Definition 14
6.2 Supervised Finetuning 14
6.3 Instruction Tuning 14
6.4 Reinforcement Learning from Human Feedback 15
6.5 Domain Adaptation 15
6.6 Benefits of Finetuning 15

Chapter 1

Introduction

Large Language Models (LLMs) such as GPT, Claude, Gemini, and LLaMA are built upon the Transformer architecture, a neural network design that has become the foundation of modern Generative Artificial Intelligence (GenAI). Transformer-based systems enable machines to understand, generate, summarize, and translate human language with unprecedented effectiveness.

Before Transformers, natural language processing systems relied heavily on recurrent neural networks (RNNs) and long short-term memory (LSTM) networks. While effective for sequential processing, these architectures struggled to capture long-range dependencies and were computationally difficult to parallelize. The introduction of the Transformer architecture fundamentally changed this landscape by replacing recurrence with attention mechanisms [2, 8].

Today, Transformers power a broad set of AI applications including chatbots, code generation systems, document summarization tools, recommendation engines, and multimodal AI systems [6].

Chapter 2

Tokenization

2.1 What is Tokenization?

Tokenization is the process of breaking text into smaller units called tokens, which helps machines process and analyze language effectively. Since computers cannot directly understand raw text, tokenization acts as the first step in transforming language into a machine-readable format [9].

For example, the sentence:

```
Transformers are powerful models.
```

may be tokenized as:

```
["Transformers", "are", "powerful", "models", "."]
```

2.2 Why Tokenization is Necessary?

Language models perform mathematical operations on numerical representations rather than words. Therefore, textual content must first be divided into manageable units that can later be mapped into numerical vectors [9].

2.3 Tokenization Approaches

2.3.1 Word Tokenization

This is the most common method where text is divided into individual words. It works well for languages with clear word boundaries, like English.

Example:

```
Machine Learning is interesting
```

becomes

```
["Machine","Learning","is","interesting"]
```

Although simple, this approach requires very large vocabularies and cannot efficiently manage unseen words.

2.3.2 Character Tokenization

In this method, text is split into individual characters. This is particularly useful for languages without clear word boundaries or for tasks that require a detailed analysis, such as spelling correction.

Example:

```
AI
```

becomes

```
["A","I"]
```

This method reduces vocabulary size but significantly increases sequence length.

2.3.3 Subword Tokenization

It strikes a balance between word and character tokenization by breaking down text into units that are larger than a single character but smaller than a full word [9].

Example:

```
Unbelievable
```

may be tokenized as

```
["Un", "believ", "able"]
```

2.3.4 Sentence tokenization

It is also a common technique used to make a division of paragraphs or large set of sentences into separated sentences as tokens [9].

2.3.5 N-gram Tokenization

N-gram tokenization splits words into fixed-sized chunks (size = n) of data [9].

2.4 Vocabulary

The collection of all supported tokens is called the model vocabulary. Different language models use different vocabulary sizes depending on their training objectives and architecture.

Chapter 3

Embeddings

3.1 From Tokens to Numerical Representations

After tokenization, language models convert tokens into numerical vectors known as embeddings. Embeddings capture semantic meaning, enabling the model to identify similarities and relationships between words and phrases [5].

3.2 One-Hot Encoding

A straightforward representation method is one-hot encoding, where each token is represented by a sparse vector containing a single non-zero value. Although simple, one-hot vectors fail to capture semantic relationships. For example, the words “car” and “vehicle” would be represented as entirely unrelated vectors.

3.3 Dense Embeddings

Transformer models use dense embedding vectors learned during training. Tokens that appear in similar contexts tend to receive similar vector representations.

Example:

- King
- Queen

- Prince
- Princess

often appear close to one another within embedding space due to their related meanings.

3.4 Positional Encoding

Unlike recurrent architectures, Transformers process every token simultaneously. Consequently, token order must be supplied explicitly through positional encodings [2, 8].

The resulting input representation is:

$$InputRepresentation = TokenEmbedding + PositionalEncoding$$

This combined representation is then passed to Transformer layers for further processing.

Chapter 4

Transformer Architecture Basics

4.1 Overview

The Transformer architecture is a deep learning framework that relies on attention mechanisms to understand relationships among tokens in a sequence. It enables efficient parallel processing while maintaining awareness of contextual dependencies [2].

4.2 How a Prompt Becomes an Output

The complete process can be summarized as:

1. User enters a prompt.
2. Prompt is tokenized.
3. Tokens are converted to embeddings.
4. Positional information is added.
5. Input passes through multiple Transformer layers.
6. Attention mechanisms identify important contextual relationships.
7. Next-token probabilities are generated.
8. A token is selected.
9. The process repeats until completion.

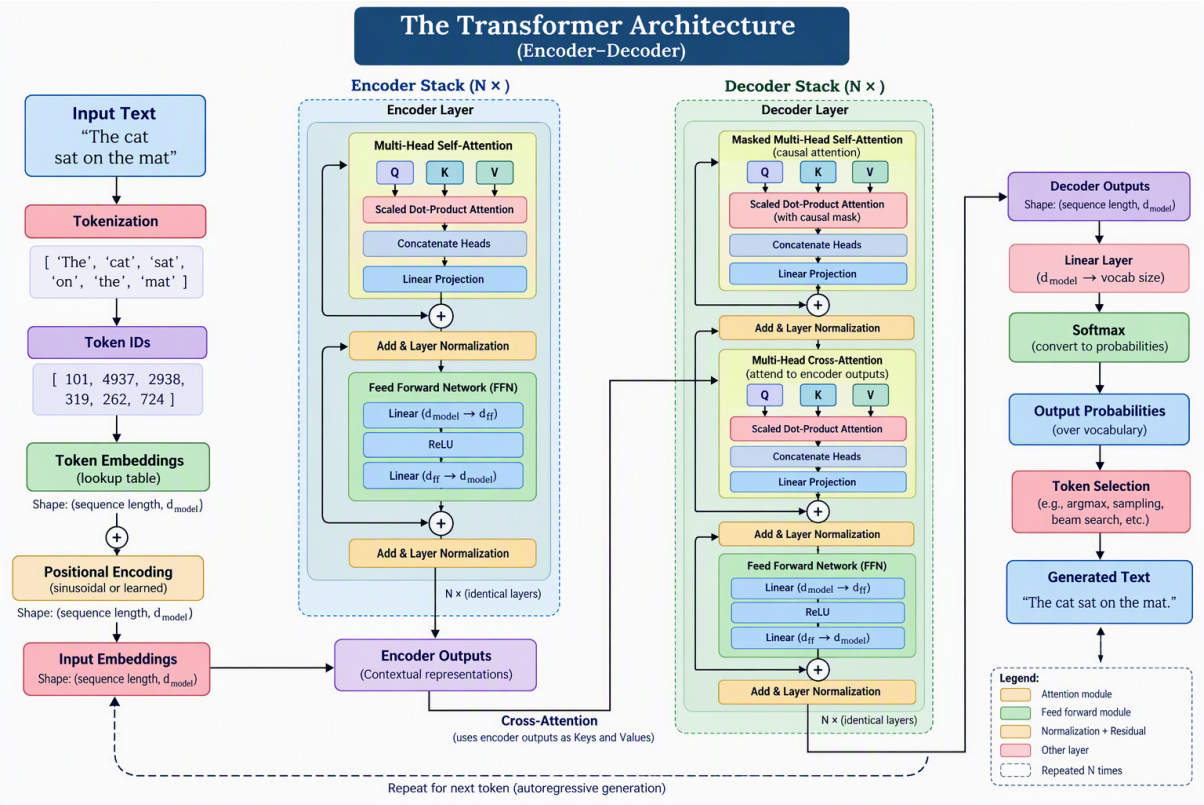


Figure 4.1: Complete Transformer Architecture showing tokenization, embeddings, positional encoding, encoder-decoder stacks, attention mechanisms, and output generation.

4.3 Self-Attention

Self-attention allows each token to examine all other tokens in the sequence and determine which are most relevant for understanding context [2].

For every token, three representations are generated:

- Query (Q)
- Key (K)
- Value (V)

The scaled dot-product attention equation is:

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k represents the dimensionality of key vectors [8].

4.4 Illustrative Example

Consider:

```
The cat sat on the mat because it was tired.
```

When processing the token “it”, the model learns to assign greater attention weights to “cat” than to unrelated words, allowing correct interpretation of the sentence meaning.

4.5 Multi-Head Attention

Instead of learning only one attention pattern, Transformers use multiple attention heads. Each head learns different contextual relationships simultaneously, allowing the model to capture different relationships and patterns in the input sequence.Instead of using a single set of Q, K, V matrices, the input embeddings are projected into multiple sets (heads), each with its own Q, K, V [3].

4.6 Feed Forward Networks

The attention output passes through a feed-forward neural network that applies nonlinear transformations to each token’s representation independently and improves representational power [2].

4.7 Residual Connections and Layer Normalization

Residual connections mechanism was introduced address the common issue of vanishing gradients. Gradients diminish as they propagate backward through numerous layers. This mechanism helps the model to learn effectively [10]. Layer normalization stabilizes training and improves convergence speed [2].

4.8 Decoder-Only Architecture

Large Language Models such as GPT use a decoder-only Transformer architecture. During text generation, tokens are allowed to attend only to previously generated tokens through causal masking, preventing the model from accessing future information [6].

4.9 Autoregressive Text Generation

Language models generate output one token at a time.

For example:

```
The capital of France is
```

The model predicts:

```
Paris
```

This new token is appended to the sequence and fed back into the model, continuing until a stopping criterion is reached [6].

Chapter 5

Pretraining

5.1 Definition

Pretraining is the large-scale learning phase during which a model is exposed to vast quantities of text data. The objective is to learn the statistical patterns, grammar, reasoning structures, and factual relationships present in language [7].

5.2 Next Token Prediction

Most modern generative language models are trained using the next-token prediction objective. For example:

```
The cat sat on the
```

The model attempts to predict the next token. Repeated across billions or trillions of examples, this objective enables language models to develop general-purpose language understanding capabilities [6].

5.3 Teacher Forcing

Training a Transformer-based language model involves predicting the next token in a sequence. During training, models commonly use a technique known as teacher forcing. In teacher forcing, the model is provided with the actual (ground-truth) previous token from the training dataset rather than its own prediction when generating the next token. This allows the model to learn the correct sequence relationships more efficiently and accelerates convergence during training.

Consider the sentence:

```
The cat sat on the mat.
```

The training process can be represented as:

Input Tokens	Target Output
The	cat
The cat	sat
The cat sat	on
The cat sat on	the
The cat sat on the	mat

During training, when predicting the token “sat”, the model receives the actual token “cat” as input rather than the token it may have predicted previously. This approach prevents error accumulation and enables more stable learning. Teacher forcing is used during the training phase of most autoregressive language models. During inference or text generation, however, the model no longer has access to the ground-truth next token and must instead use its own previously generated tokens to continue generation. This difference between training and inference is commonly referred to as **exposure bias**. Teacher forcing significantly improves training efficiency and remains a fundamental technique for training modern Transformer-based language models.

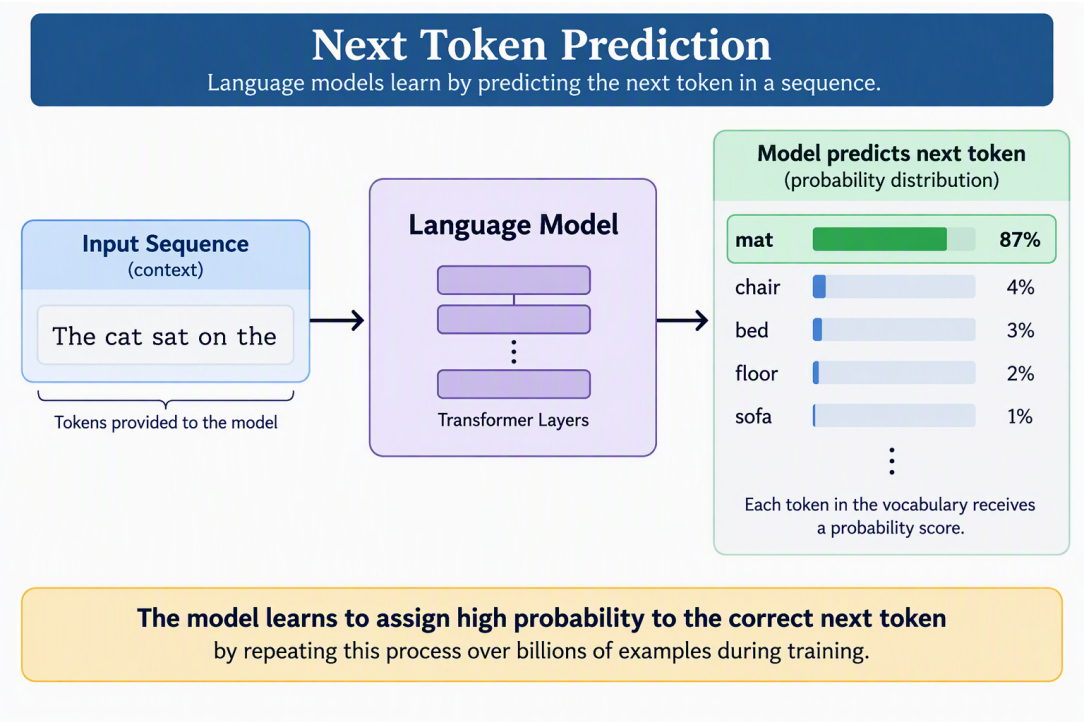


Figure 5.1: Overview of next-token prediction in Transformer-based language models.

5.4 Training Data Sources

Pretraining datasets may contain:

- Books
- Web pages
- Research papers
- Technical documentation
- Encyclopedias
- Public repositories

Large-scale and diverse datasets improve the generalization capability of foundation models [7].

5.5 Computational Requirements

Pretraining modern LLMs requires large-scale GPU or TPU clusters, substantial storage infrastructure, and distributed training frameworks capable of handling billions of model parameters [2].

Chapter 6

Finetuning

6.1 Definition

Finetuning is the process of adapting a pretrained model to particular tasks, domains, or desired behaviors by providing additional specialized training data [1].

6.2 Supervised Finetuning

Supervised Finetuning (SFT) uses example input-output pairs to teach the model task-specific behavior.

Example:

```
Input: Explain cloud computing.
Output: Cloud computing refers to...
```

Through repeated exposure to examples, the model learns task-specific response patterns [1].

6.3 Instruction Tuning

Instruction tuning extends supervised finetuning by teaching the model how to understand and follow natural language instructions from users [1].

6.4 Reinforcement Learning from Human Feedback

RLHF aligns language models with human preferences by incorporating rankings and feedback collected from human evaluators. A reward model is trained from this feedback and then used to optimize model behavior [4].

6.5 Domain Adaptation

Organizations may finetune models for specific domains:

- Healthcare
- Finance
- Law
- Software Engineering
- Customer Support

This process improves accuracy and relevance within specialized environments [1].

6.6 Benefits of Finetuning

Finetuning enables organizations to make use of the broad knowledge acquired during pretraining while adapting the model to specialized business or research objectives [6].

Bibliography

[1] Hugging Face. *Fine-Tuning Transformers*. 2024. URL: https://huggingface.co/docs/transformers/training.

[2] IBM. *What are Transformers in Machine Learning?* 2024. URL: https://www.ibm.com/topics/transformer-model.

[3] *Multi-Head Attention Mechanism*. URL: https://www.geeksforgeeks.org/nlp/multi-head-attention-mechanism/.

[4] OpenAI. *Aligning Language Models to Human Preferences*. 2024. URL: https://openai.com/research.

[5] OpenAI. *Embeddings Guide*. 2024. URL: https://platform.openai.com/docs/guides/embeddings.

[6] OpenAI. *GPT-4 Technical Report*. 2023. URL: https://arxiv.org/abs/2303.08774.

[7] Stanford CRFM. *Foundation Models Overview*. 2024. URL: https://crfm.stanford.edu.

[8] Ashish Vaswani et al. "Attention Is All You Need". In: *NeurIPS* (2017). URL: https://arxiv.org/abs/1706.03762.

[9] *What is tokenization?* URL: https://www.geeksforgeeks.org/nlp/what-is-tokenization/.

[10] *Why Are Residual Connections Important in Transformer Architectures?* URL: https://www.baeldung.com/cs/transformer-networks-residual-connections.